

Where Post-Training Quantization Breaks Agentic Tool Calling: A k-Quant Ladder on Qwen3 and Llama

Anonymous authors

Paper under double-blind review

Abstract

Models reach laptops through post-training quantization, and the format that ships is the llama.cpp k-quant. Its cost has been measured on prose, knowledge and reasoning, never on what an agent does, which is emit a function call that a program executes. We build a controlled ladder, five k-quant formats from one FP16 source, and score each rung on the Berkeley Function Calling Leaderboard by deterministic syntax-tree matching, with no model judging another anywhere in the pipeline. We pre-registered the intuition that a function signature, having no slack, would break at a higher bit-width than free-form text. It does not. On Qwen3-1.7B tool calling is flat from Q8_0 through Q4_K_M and loses 4.75 points at Q3_K_M, about 10 times the run-to-run noise we measure by re-running identical weights. On the same weights, free-form mathematics loses 39.50 points at the same rung, because 53% of generations run out of a 4,096-token budget still inside the chain of thought. Tool calls are short enough that this run-away never has room to happen, which is why the schema survives. The cliff is absent on Qwen3-8B and Qwen3-14B. A second family does not follow the pattern: Llama-3.2-3B already loses 4.00 points at Q4_K_M, and Llama-3.1-8B scores 21.50 points higher at Q3_K_M than at Q8_0 under the strict checker because its full-precision weights quote numeric arguments as strings; a checker that coerces such strings restores the expected order and a 7.00-point loss at three bits. A fair answer extractor changes the free-form numbers by at most 0.50 points, and a logit penalty on overthinking markers has no measurable effect on tool calling at either rung. On Qwen3, four-bit quantization is safe and three-bit is not; on Llama the floor moves with the model, so the bit-width an agent can tolerate has to be measured rather than assumed, and the damage quantization does to long reasoning does not carry over to the tool call.

1 Introduction

Language models reach laptops by being compressed. The weights are stored in fewer bits than they were trained in, and the file shrinks accordingly. The full-precision file for Qwen3-1.7B is 3.8 GB; the version most people would actually run, Q4_K_M, is 1.3 GB, about 34% of it. That compression is post-training quantization (Dettmers et al., 2022; Frantar et al., 2023; Lin et al., 2024), and the format nearly everyone uses on a laptop is the family of llama.cpp k-quants (Gerganov & contributors, 2023), because that is what Ollama and every desktop front end serve. Four bits is where the scaling analysis says the trade-off is best (Dettmers & Zettlemoyer, 2023; Kumar et al., 2025), and it is the default most front ends pull. Lower bit-widths are pursued for throughput as well as size, although Malekar & Zand (2025) show, for binary and ternary weights, that those gains are capped by whatever part of the model stays in higher precision.

The cost of that compression has been measured with care, but on the wrong tasks for one growing use. Broad evaluations cover language modelling, classification, knowledge, dialogue, long context and trustworthiness across many families (Jin et al., 2024; Li et al., 2024), multilingual degradation has its own study (Marchisio et al., 2024), long-context inputs lose up to 59% under four-bit methods (Mekala et al., 2025), and reasoning models have been quantized and scored on mathematics and code (Liu et al., 2025). Kurt (2026) quantizes Llama-3.1-8B-Instruct into thirteen GGUF formats and scores each on GSM8K, HellaSwag, IFEval, MMLU, TruthfulQA and perplexity. Lotfi et al. (2026) measure five reasoning models under GPTQ, AWQ and

FlatQuant on mathematics, code and science questions and find accuracy falling as chain-of-thought inflates. Every task in both papers is free-form or multiple choice, and none of them involves emitting a function call, which is the output an agent actually produces.

An agent calls tools (Patil et al., 2024; Qin et al., 2024; Yao et al., 2025). It emits a function name and a set of typed arguments that a program will execute. That output either parses against the schema or it does not, with no partial credit and no reader to accept an approximate answer. Our starting intuition was that a function signature has none of the slack prose has, and that compression should therefore break tool calling at a higher bit-width than free-form accuracy. We pre-registered this as our first hypothesis before running anything.

We build a controlled ladder, five k-quant formats produced from one FP16 source, and score each rung on the Berkeley Function Calling Leaderboard (Patil et al., 2025), which compares a model’s call to a reference by deterministic abstract-syntax-tree matching. No model judges another model anywhere in the pipeline. We then run the same weights on a free-form control so that the two floors can be compared on identical parameters.

For Qwen3-1.7B the answer is the reverse of the one we expected. Tool-calling accuracy is flat from Q8_0 through Q4_K_M and falls by 4.75 points at Q3_K_M, roughly 10 times the run-to-run noise of the instrument. On the same weights free-form mathematics falls by 39.50 points at the same rung. Both floors sit between four and three bits, but the drop in tool calling is about an eighth of the drop in mathematics. The difference comes down to output length. At three bits the model’s chain of thought (Wei et al., 2022) grows until 53% of math generations use up their token budget before reaching an answer. A tool call is a median of 182 tokens, so the same weights never get the chance to wander. On Qwen3-8B and Qwen3-14B there is no cliff at any rung. On a second family the picture changes: Llama-3.2-3B already loses at four bits, and Llama-3.1-8B scores higher on tool calling when compressed, for a reason that has nothing to do with capability (§4.5).

Contributions. A controlled k-quant ladder for agentic tool calling, built from one source per model so that only the weights change between rungs (§3). A reproducibility measurement that most quantization papers omit: the same weights run three times, which bounds what a ladder difference can mean (§4.1). The floor for Qwen3-1.7B, located between Q4_K_M and Q3_K_M by paired testing (§4.2). The free-form comparison on identical weights, which reverses H1 and locates the mechanism in chain-of-thought length rather than in the schema (§4.3). The same measurement at 8B and 14B, where the cliff disappears (§4.4). The same ladder on a second family, Llama-3.2-3B and Llama-3.1-8B, where the floor sits above four bits for the 3B model and where compression raises the 8B model’s schema score by changing an argument-typing habit (§4.5). A deployment table for a question practitioners currently answer by guessing.

2 Related work

Quantization evaluated on free-form tasks. The broad evaluations of post-training quantization score many model families on language modelling, multiple-choice knowledge, dialogue, long-context and trustworthiness tasks (Jin et al., 2024; Li et al., 2024); a full-text search of both finds no tool-calling or function-calling benchmark. Liu et al. (2025) quantize the DeepSeek-R1-distilled and QwQ reasoning models and find mathematics and code degrading under aggressive weight and KV-cache quantization, again on free-form tasks. Kurt (2026) is the closest work and the one this paper extends. One model, thirteen llama.cpp formats from Q3_K_S to Q8_0 against FP16, six free-form or multiple-choice benchmarks, plus throughput and size. A full-text read finds no tool-calling, function-calling, agentic, structured-output or schema-compliance evaluation, and the conclusion asks for exactly what we add: additional model families and sizes under the same protocol. Lotfi et al. (2026) take a different axis, in the line of work on overthinking in reasoning models (Chen et al., 2024; Sui et al., 2025). Under GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024) and FlatQuant, five reasoning models lose accuracy while emitting longer chains of thought, and in up to 52% of failures the correct answer appears mid-trace without being committed to. Two details matter for the comparison. Their compression methods are research methods rather than the k-quants that ship to laptops,

and their failure categorisation, including the overthinking class, relies on GPT-5 as a judge, whereas our pipeline contains none.

Tool-calling evaluation. Gorilla (Patil et al., 2024) and ToolLLM (Qin et al., 2024) established API calling as a benchmark task, and τ -bench (Yao et al., 2025) extends it to multi-turn agent-user interaction. BFCL (Patil et al., 2025) scores a model’s function call against a reference by abstract-syntax-tree matching over the function name, argument names, types and values, across single, multiple and parallel calls, an irrelevance category where no call is correct, and multi-turn trajectories. Because the matcher is deterministic, the same generation always receives the same verdict, which is what lets a ladder difference be attributed to the weights rather than to the scorer. Dondeti (2026) study the same class of small models on consumer hardware from the safety side, scoring adversarial tool calls from seven models between 1B and 4B parameters with a deterministic would-dispatch rule; this paper is the accuracy side of the same deployment.

The gap. Quantization has been scored on prose, knowledge, instruction following and reasoning, both for research compression methods and for the deployed one. It has not been scored on tool calling under either, and that is what we measure.

3 Method

Pre-registration. Hypotheses and thresholds were fixed on 18 September 2026 before the first grid run and are in `paper/preregistration.md`. Two full-text summaries were appended to that file during the first rung as documentation; no hypothesis or threshold changed after runs began.

The ladder. Every rung is produced from a single FP16 GGUF of Qwen3-1.7B (Qwen Team, 2025) with `llama-quantize`, so the rungs differ in weights and in nothing else. The formats are a subset of the ladder in Kurt (2026), chosen to span the range at one rung per bit-width: Q8_0, Q6_K, Q5_K_M, Q4_K_M and Q3_K_M. Ollama’s library publishes only three of these for this model, which is why the ladder is built rather than pulled.

Serving. Each rung is served by `llama-server` with full Metal offload under a fixed model alias, so the harness sends identical requests to every rung. The single-call ladder and the repeats use four slots and an 8,192-token context; the free-form control, the larger models and the multi-turn runs use 32,768, because reasoning traces overflowed the smaller context; the 16,384-token budget re-run uses 65,536 with two slots. Generation is at temperature 0.001. Measured throughput is about 35 tokens per second per slot and 650 tokens per second of prompt evaluation on an Apple M1 Max.

Scoring. BFCL v4, `simple_python` category, 400 questions, deterministic AST matching. Differences between rungs are tested pairwise on the same 400 questions with McNemar’s exact test (McNemar, 1947), which uses only the discordant pairs and is the appropriate test when two conditions are scored on identical items.

Reproducibility control. Two rungs, Q8_0 and Q4_K_M, were run three times each with identical weights and settings. This gives the run-to-run variation of the instrument, which is needed before any difference between rungs can be read as a difference between weights.

Free-form control. The same five weight files answer 400 GSM8K test questions (Cobbe et al., 2021), drawn at seed 42, through the same server under the same alias, with a 4,096-token completion budget. Every generation is logged in full so that scoring under a different extractor is a re-read rather than a re-run. Two extractors were fixed before any rung was scored. Strict accepts an explicit commitment in any of three forms: “The answer is N ”, “#### N ”, or `\boxed{N}`. Lenient takes the last number in the visible answer. We record that our first strict rule accepted only the first form and would have scored 21 of the first 59 generations wrong for writing the answer in a box; that rule was a strawman and was replaced before scoring, and the gap it would have produced is not reported anywhere in this paper.

Rung	Accuracy per run	Range (pp)	Max per-question flips
Q8_0	91.75, 91.50, 91.50	0.25	1 of 400
Q4_K_M	92.00, 91.50, 92.00	0.50	4 of 400

Table 1: Run-to-run variation with identical weights and settings, `simple_python`, $n = 400$.

Rung	Size (GB)	Accuracy [95% CI]	vs. Q8_0	Lost	Gained	McNemar p
Q8_0	2.2	91.75% [88.6, 94.1]	baseline			
Q6_K	1.7	93.50% [90.6, 95.5]	+1.75 pp	6	13	0.167
Q5_K_M	1.5	90.75% [87.5, 93.2]	−1.00 pp	10	6	0.454
Q4_K_M	1.3	92.00% [88.9, 94.3]	+0.25 pp	13	14	1.000
Q3_K_M	1.1	87.00% [83.3, 89.9]	−4.75 pp	30	11	0.004

Table 2: The quantization ladder on `simple_python`, $n = 400$ per rung, all rungs from one FP16 source. Lost and gained count questions that flip against Q8_0; p is McNemar’s exact test on those discordant pairs.

Larger models and a second family. Qwen3-8B, same three rungs Q8_0, Q4_K_M and Q3_K_M from its own FP16 source, on the first 200 questions of the same category in file order, and Qwen3-14B under the identical protocol. Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct, whose Llama 3.1 models are described in Llama Team, AI @ Meta (2024), from their own FP16 sources, at the same three rungs, on the same 200 questions and the same 400 GSM8K questions with the same prompt and budget. Both Llama models are instruction-tuned rather than reasoning-tuned, which is deliberate: it separates what the ladder does to a tool call from what it does to a chain of thought. Llama-3.1-8B-Instruct is also the model Kurt (2026) quantized, so the two ladders meet on one model.

Overthinking penalty. Lotfi et al. (2026) report that a fixed logit penalty on a curated list of overthinking markers cuts chain-of-thought length by 12 to 23% while preserving accuracy. We apply a fixed penalty of -2.0 to the eight Qwen3 tokens for *wait*, *but*, *alternatively* and *hmm* in both capitalisations, at the server, and re-run the Qwen3-1.7B ladder at Q4_K_M and Q3_K_M. This approximates their method with a single fixed λ rather than their tuned value.

4 Results

4.1 The instrument is reproducible

Table 1 gives the same weights run three times. Q8_0 moved by 0.25 points across three runs and Q4_K_M by 0.50. At the level of individual questions, at most 1 of 400 verdicts changed between two Q8_0 runs and at most 4 between two Q4_K_M runs. The instrument is close to deterministic, so any difference between rungs larger than about 0.50 points can be attributed to the weights rather than to run-to-run variation.

4.2 Flat to four bits, a cliff at three

Table 2 is the ladder. From Q8_0 down to Q4_K_M the accuracy sits in a band of under three points and no rung differs from Q8_0 at any conventional level: Q4_K_M is at 92.00% against Q8_0’s 91.75%, a difference of +0.25 points with a McNemar p of 1.000, while being 59% of the file size. Q6_K scores above Q8_0 and Q5_K_M below it, and neither difference survives paired testing, so the apparent non-monotonicity in the raw numbers is within the instrument’s noise.

Q3_K_M is a different case. It loses 30 questions that Q8_0 gets right and recovers 11, a net loss of 4.75 points at $p = 0.004$. That drop is about 10 times the largest run-to-run range in Table 1, which places the tool-calling floor for this model between four and three bits.

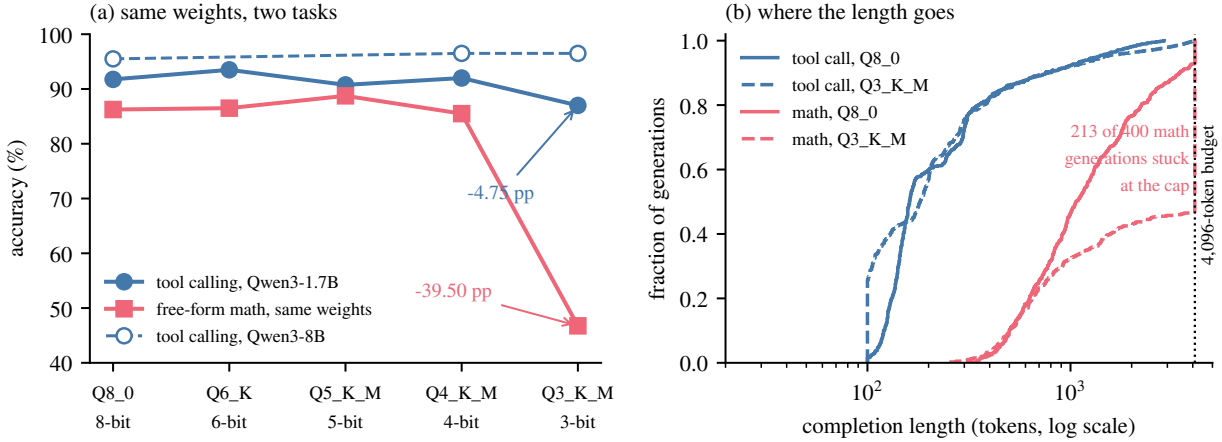


Figure 1: (a) Accuracy across the quantization ladder on identical weight files: tool calling on BFCL and free-form mathematics on GSM8K for Qwen3-1.7B, with Qwen3-8B tool calling dashed. Both floors sit at Q3_K_M; the free-form drop is about eight times the tool-calling drop. (b) Cumulative distribution of completion length at Q8_0 and Q3_K_M for each task. Tool calls are short at both rungs, while math generations at Q3_K_M pile up against the 4,096-token budget and 213 of 400 never finish.

Rung	Strict [95% CI]	Lenient	Extractor gap	vs. Q8_0 (strict)	McNemar p	Median tokens
Q8_0	86.25% [82.5, 89.3]	86.50%	+0.25 pp	baseline		1065
Q6_K	86.50% [82.8, 89.5]	86.50%	+0.00 pp	+0.25 pp	1.000	1102
Q5_K_M	88.75% [85.3, 91.5]	89.25%	+0.50 pp	+2.50 pp	0.164	955
Q4_K_M	85.50% [81.7, 88.6]	85.75%	+0.25 pp	-0.75 pp	0.736	1048
Q3_K_M	46.75% [41.9, 51.6]	46.75%	+0.00 pp	-39.50 pp	<0.0001	4096

Table 3: The free-form control, GSM8K, $n = 400$ per rung, identical weight files to Table 2. Extractor gap is lenient minus strict. Median tokens is the completion length; the budget was 4,096.

4.3 The free-form floor on the same weights

Figure 1 puts both tasks on one axis and Table 3 gives the numbers. On identical weights the tool-calling line barely moves while the free-form line drops sharply at the same rung, and the right panel shows the reason. The free-form floor sits exactly where the tool-calling floor sits, between Q4_K_M and Q3_K_M, and nowhere else: no rung above Q3_K_M differs from Q8_0 under paired testing. So H1 as pre-registered fails: the schema does not break at a higher bit-width than prose. It fails in the opposite direction. At Q3_K_M free-form accuracy falls by 39.50 points, against 4.75 for tool calling on identical weights, so the output with no slack is the one that holds up.

The mechanism is in the last two columns of the table. At Q3_K_M, 213 of 400 generations hit the 4,096-token completion budget while still inside the chain of thought and never emit an answer. The median completion length at this rung is the budget itself. This is the behaviour Lotfi et al. (2026) report under GPTQ and AWQ, reproduced here on the deployed format with no judge in the loop. Among the 187 generations that did finish, accuracy is 84.5%, against 92.7% for finished generations at Q8_0, so a smaller genuine reasoning loss of about 8.2 points sits underneath the collapse. A tool call does not pay that cost. At Q3_K_M the median tool-call generation is 182 tokens, the 90th percentile 753, and only 1 of 539 requests came near the budget. The generation ends before the same instability has room to develop.

The extractor is not the story. The gap between strict and lenient scoring is at most 0.50 points at any rung. H3 predicted a gap of at least five points that would bound how much published free-form degradation is the parser. Once the strict extractor accepts the commitment format that reasoning models actually emit,

Model	Rung	n	Strict [95% CI]	vs. Q8_0	p	Coerced [95% CI]	vs. Q8_0	p
Qwen3-1.7B	Q8_0	400	91.75% [88.6, 94.1]	baseline		91.75% [88.6, 94.1]	baseline	
Qwen3-1.7B	Q4_K_M	400	92.00% [88.9, 94.3]	+0.25 pp	1.000	92.00% [88.9, 94.3]	+0.25 pp	1.000
Qwen3-1.7B	Q3_K_M	400	87.00% [83.3, 89.9]	-4.75 pp	0.004	87.00% [83.3, 89.9]	-4.75 pp	0.004
Qwen3-8B	Q8_0	200	95.50% [91.7, 97.6]	baseline		95.50% [91.7, 97.6]	baseline	
Qwen3-8B	Q4_K_M	200	96.50% [93.0, 98.3]	+1.00 pp	0.625	96.50% [93.0, 98.3]	+1.00 pp	0.625
Qwen3-8B	Q3_K_M	200	96.50% [93.0, 98.3]	+1.00 pp	0.688	96.50% [93.0, 98.3]	+1.00 pp	0.688
Qwen3-14B	Q8_0	200	96.00% [92.3, 98.0]	baseline		96.00% [92.3, 98.0]	baseline	
Qwen3-14B	Q4_K_M	200	96.00% [92.3, 98.0]	+0.00 pp	1.000	96.00% [92.3, 98.0]	+0.00 pp	1.000
Qwen3-14B	Q3_K_M	200	96.50% [93.0, 98.3]	+0.50 pp	1.000	96.50% [93.0, 98.3]	+0.50 pp	1.000
Llama-3.2-3B	Q8_0	200	92.00% [87.4, 95.0]	baseline		93.50% [89.2, 96.2]	baseline	
Llama-3.2-3B	Q4_K_M	200	88.00% [82.8, 91.8]	-4.00 pp	0.021	91.00% [86.2, 94.2]	-2.50 pp	0.125
Llama-3.2-3B	Q3_K_M	200	86.50% [81.1, 90.6]	-5.50 pp	0.007	91.00% [86.2, 94.2]	-2.50 pp	0.180
Llama-3.1-8B	Q8_0	200	50.00% [43.1, 56.9]	baseline		92.00% [87.4, 95.0]	baseline	
Llama-3.1-8B	Q4_K_M	200	63.00% [56.1, 69.4]	+13.00 pp	<0.0001	90.50% [85.6, 93.8]	-1.50 pp	0.453
Llama-3.1-8B	Q3_K_M	200	71.50% [64.9, 77.3]	+21.50 pp	<0.0001	85.00% [79.4, 89.3]	-7.00 pp	0.003

Table 4: Tool calling across model size, `simple_python`. Qwen3-1.7B at $n = 400$; Qwen3-8B and Qwen3-14B at $n = 200$. Each model is compared against its own Q8_0.

no such gap exists. The failures at Q3_K_M are generations that contain no answer at all rather than answers a parser failed to find.

Nor is the budget. A collapse driven by truncation could be an artefact of the 4,096-token cap rather than of the weights. We re-ran the 213 Q3_K_M generations that hit the cap with a budget of 16,384 tokens, four times the original. Of the 196 that completed without a server error, 151 (77%) hit the larger cap as well, and only 48 (24%) reached a correct answer. Given four times the room, the three-bit model mostly keeps thinking. The run-away is a property of the weights, and a larger budget converts most of the truncations into longer truncations.

4.4 Across models

Table 4 adds Qwen3-8B at the three rungs that matter, and at 8B there is no cliff. Q3_K_M scores 96.50% against 95.50% at Q8_0, a difference of +1.00 points at $p = 0.688$, and Q4_K_M is likewise indistinguishable from the baseline. Qwen3-14B repeats the pattern: 96.00% at Q8_0, 96.00% at Q4_K_M and 96.50% at Q3_K_M, with the Q3_K_M difference at +0.50 points and $p = 1.000$. On this family the three-bit floor that binds at 1.7B has moved below the ladder by 8B and stays there at 14B, which is the direction the free-form literature reports for larger models and is now shown for the call itself at two sizes.

4.5 A second family

Table 5 repeats the three-rung ladder on two Llama models. The Qwen3 pattern does not transfer. Llama-3.2-3B loses 4.00 points on tool calling at Q4_K_M ($p = 0.021$) and 5.50 at Q3_K_M ($p = 0.007$), so on this model the floor is already above four bits. Its free-form accuracy falls 8.50 points at Q3_K_M, and again the tool call is the smaller loss, but the margin between the two tasks is narrower than on Qwen3-1.7B.

Llama-3.1-8B moves the other way under the strict checker. Its Q8_0 weights score 50.00% on tool calling; Q4_K_M and Q3_K_M score 63.00% and 71.50%, both at $p < 0.0001$ against Q8_0. The failure breakdown gives the reason. Of the 100 Q8_0 failures, 90 are type errors: the model names the right function and the right value but quotes the number, as in `{"base": "10"}`, and the AST checker rejects the type. At Q3_K_M, 32 of 57 failures are of that kind. To separate the habit from the capability we re-score every rung under a coercing checker that accepts a quoted number, list or boolean when the parsed value is what the reference holds; nothing is re-run, and the Qwen3 scores do not change because none of their failures are

Model	Rung	Tool acc.	vs. Q8_0	p	Coerced	GSM8K strict	vs. Q8_0	p
Llama-3.2-3B	Q8_0	92.00%	baseline		93.50%	84.00%	baseline	
Llama-3.2-3B	Q4_K_M	88.00%	-4.00 pp	0.021	91.00%	81.25%	-2.75 pp	0.126
Llama-3.2-3B	Q3_K_M	86.50%	-5.50 pp	0.007	91.00%	75.50%	-8.50 pp	<0.0001
Llama-3.1-8B	Q8_0	50.00%	baseline		92.00%	91.00%	baseline	
Llama-3.1-8B	Q4_K_M	63.00%	+13.00 pp	<0.0001	90.50%	87.75%	-3.25 pp	0.041
Llama-3.1-8B	Q3_K_M	71.50%	+21.50 pp	<0.0001	85.00%	83.25%	-7.75 pp	<0.0001

Table 5: The second family. Tool calling on `simple_python`, $n = 200$ per rung; GSM8K strict accuracy, $n = 400$; each model against its own Q8_0 by McNemar’s exact test.

of this type. The coerced column of Table 5 restores the expected order: Llama-3.1-8B scores 92.00%, 90.50% and 85.00% down the ladder, a 7.00-point loss at three bits, and Llama-3.2-3B scores 93.50%, 91.00% and 91.00%. So the compressed 8B weights write bare numbers more often, which raises the strict score, while the underlying call gets slightly worse and free-form accuracy falls 7.75 points at the same rung. Length plays no part in either model: GSM8K completions have a median of 185 to 220 tokens, and at most 5 of 400 reach the budget at any rung, so the run-away behind the Qwen3 collapse cannot occur and the free-form losses here are ordinary precision loss.

Two readings follow. The tool-calling floor is a property of the model, not of the format: four bits is safe on every Qwen3 size we measured and costs Llama-3.2-3B 4.00 points under the strict checker and 2.50 under the coercing one. And a strict schema score can move in either direction under quantization, because it measures a formatting habit as much as a capability; anyone scoring tool calls under compression should report the failure types and a coerced score beside the strict one.

4.6 Harder categories

The pre-registration predicted that multi-turn trajectories, where an error in one call corrupts the state for every call after it, would degrade at least twice as fast as single calls. They degrade far faster than that. On BFCL’s `multi_turn_base`, $n = 200$, Qwen3-1.7B scores 17.00% at Q8_0, 13.50% at Q4_K_M and 1.50% at Q3_K_M. The Q4_K_M drop is -20.6% relative against +0.3% for single calls at the same rung, in the predicted direction but not significant at this n ($p = 0.230$). The Q3_K_M drop is -91.2% relative against -5.2% for single calls, at $p < 0.0001$. H5 is supported at three bits by a wide margin.

The size of the multi-turn collapse follows from §4.3. A single call is a short generation and survives three-bit weights because it ends before the length instability has room to develop. A multi-turn trajectory is many generations in sequence, each conditioned on the accumulated state, so the same per-step failure rate compounds and a single malformed call anywhere in the sequence fails the whole trajectory. The server logs show the instability reaching the call once calls are chained: at Q3_K_M the median generation per request is 509 tokens against 213 at Q8_0, and the 90th percentile 2403 against 773. The same weights that keep a single call to 182 tokens let a conditioned call run more than twice as long, and the extra length is the run-away from §4.3, now with a schema to break. The low absolute base rate, 17.00% even at Q8_0, is a property of the category for a 1.7B model and is why the Q4_K_M comparison is underpowered; it does not affect the Q3_K_M conclusion.

4.7 Does the overthinking penalty transfer

The penalty has no measurable effect. With it, Qwen3-1.7B scores 92.00% at Q4_K_M and 87.00% at Q3_K_M, identical to the unpenalised runs to two decimals; at each rung 2 questions flip one way and 2 the other. H4 predicted a loss of at least three points and there is no loss. The explanation follows from §4.3: the penalty acts on tokens a reasoning trace emits and a tool call does not, so on a 182-token output there is almost nothing for it to act on. This agrees with the authors’ own scope statement that the marker list was curated for free-form English reasoning. Both the failure they describe and the fix for it live in the reasoning trace, and the tool call contains very little of that trace.

5 Limitations

Three sizes of one model family and two sizes of a second, on the easiest BFCL category plus one multi-turn set for Qwen3-1.7B only, on one hardware platform. The ladder is one quantization family; GPTQ, AWQ and the other research methods are deliberately out of scope, and multilingual degradation is excluded because Marchisio et al. (2024) already cover it. The reproducibility control was run on two rungs rather than all five. The free-form control uses a 4,096-token budget; the 16,384-token re-run in §4.3 shows that a larger budget recovers few of the 213 truncated generations. The overthinking penalty is one fixed λ rather than the authors’ tuned value. Multi-turn is underpowered at $n = 200$ and a 17% base rate, and was not run on Llama. A third family, and the multi-turn category on Llama, are the next additions.

6 Conclusion

We expected a function signature, having no slack, to be the first thing quantization breaks. On the deployed format, with no judge in the pipeline, it turned out to be the output that held up best. For Qwen3-1.7B the tool-calling floor and the free-form floor sit at the same rung, between four and three bits, but the call loses 4.75 points where the mathematics loses 39.50. The difference is explained by length. Three-bit weights make the chain of thought grow past the token budget, and a call is complete long before that happens. On Qwen3-8B and Qwen3-14B there is no tool-calling cliff at any rung. On Llama the floor is elsewhere: Llama-3.2-3B loses at four bits, and Llama-3.1-8B gains at three because compression changes how it types its arguments. A fair extractor changes nothing, and a penalty designed for reasoning traces finds little to act on in a call. For anyone deploying an agent on a laptop, the practical reading is that four bits is safe on Qwen3 and not on Llama-3.2-3B, that the bit-width has to be measured on the model being shipped, and that the reasoning-length failure described in the quantization literature stays in the reasoning and does not reach the tool call.

The measurement we would most like other quantization studies to adopt is the reproducibility control in Table 1. Identical weights re-run three times moved by at most 0.50 points. Without that figure, none of the other differences in this paper could be attributed to anything.

Reproducibility

Every number in this paper is a macro recomputed from the raw BFCL result and score files, and nothing is typed by hand. All raw generations are retained.

- `analysis/paper_numbers.py` writes `paper/tables/numbers.tex` and both tables from `runs/`.
- `run_ladder.sh` rebuilds the ladder from the FP16 source and scores it; `run_model.sh` does the same for Qwen3-8B, Qwen3-14B and the multi-turn category.
- `run_repeats.sh`, `run_gsm8k.sh`, `run_budget.sh` and `run_h4.sh` produce the reproducibility, free-form, budget and penalty arms.

Code, run logs, raw generations and the pre-registration are provided as an anonymized archive in the supplementary material, at the release tag the paper describes; the public repository will be linked on acceptance. The GGUF weight files are not in the archive; `run_ladder.sh` rebuilds them from the published FP16 checkpoint.

Broader Impact Statement

This paper measures how a widely used compression format changes the accuracy of small language models at emitting function calls. The direct use is to let practitioners pick a bit-width for on-device agents from measurements rather than guesswork. A model that emits malformed or wrong tool calls can take real actions, so the multi-turn collapse at three bits reported in §4.4 is a reason to test agents at the exact

quantization they ship with. We see no risk of harm arising from the paper beyond what already attaches to deploying tool-calling agents.

References

- Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do NOT Think That Much for $2+3=?$ On the Overthinking of o1-Like LLMs. *arXiv preprint arXiv:2412.21187*, 2024.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit Inference Scaling Laws. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems 35*, 2022.
- Ravitez Dondeti. ToolGuard: Red-Teaming Small Language Model Tool Calling on Consumer Hardware. *Transactions on Machine Learning Research*, 2026.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations*, 2023.
- Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.
- Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A Comprehensive Evaluation of Quantization Strategies for Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling Laws for Precision. In *International Conference on Learning Representations*, 2025.
- Uygar Kurt. Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct. *arXiv preprint arXiv:2601.14277*, 2026.
- Shiyao Li, Xuefei Ning, Luning Wang, Tengxuan Liu, Xiangsheng Shi, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Evaluating Quantized Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Machine Learning and Systems (MLSys)*, 2024.
- Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models. In *Conference on Language Modeling (COLM)*, 2025.
- Llama Team, AI @ Meta. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.
- Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. Quantized Reasoning Models Think They Need to Think Longer, but They Do Not. *arXiv preprint arXiv:2606.00206*, 2026.
- Jinendra Malekar and Ramtin Zand. Amdahl’s law for LLMs: A throughput-centric analysis of extreme LLM quantization. *Transactions on Machine Learning Research*, 2025.

- Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. How Does Quantization Affect Multilingual LLMs? In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- Quinn McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2):153–157, 1947.
- Anmol Mekala, Anirudh Atmakuru, Yixiao Song, Marzena Karpinska, and Mohit Iyyer. Does Quantization Affect Models’ Performance on Long-Context Tasks? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 9422–9470, 2025. doi: 10.18653/v1/2025.emnlp-main.479.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems 37*, 2024.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 48371–48392, 2025.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, 2024.
- Qwen Team. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models. *Transactions on Machine Learning Research*, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35*, 2022.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations*, 2025.